

Auditoría de Código KognitoAI

Auditoría de Código - KognitoAI

Fecha: 2026-07-30

Agente: KogniTerm (Orquestador Principal)

Alcance: Análisis estático, seguridad, arquitectura y patrones

Metodología: Radon, Bandit, grep estático, revisión manual de archivos críticos

1. Resumen Ejecutivo

Métrica	Valor
Líneas de código Python	43,961
Archivos Python analizados	~200
Complejidad Ciclomática máxima	313 (core/agent.py::call_model_node)
Índice de Mantenibilidad mínimo	0.00 (Rango C - Crítico)
Vulnerabilidades High	7
Vulnerabilidades Medium	26
Vulnerabilidades Low	55
Vulnerabilidades previas sin corregir	16 (8+ meses)

Conclusión: El proyecto presenta **riesgo crítico de mantenibilidad y seguridad**. Existen módulos con complejidad extrema, secretos embebidos por defecto, endpoints de terminal sin aislamiento real y múltiples inyecciones SQL potenciales. Se requieren acciones inmediatas en los módulos core/agent.py, api/terminal.py y core/config.py.

2. Análisis de Complejidad y Mantenibilidad

2.1 Archivos Críticos por Complejidad

Archivo	CC Promedio	MI	Rango	Función Más Compleja	C
core/agent.py	18.76	0.00	C - Crítico	call_model_node	3
core/autonomous_heartbeat.py	12.55	0.00	C - Crítico	run_custom_user_heartbeat	3
core/memory_manager.py	12.55	N/A	Alto	_run_semantic_search	3
core/llm_manager.py	11.22	N/A	Alto	initialize_llms	1
api/analysis.py	7.69	0.00	C - Crítico	get_all_analysis_endpoint	1

2.2 Código Smells Identificados

1. **God Object:** core/agent.py con 3,000+ líneas y CC 313 en una sola función.
2. **Long Method:** call_model_node supera los límites aceptables de complejidad.
3. **高 complejidad en lógica de negocio:** initialize_llms (CC 111) mezcla configuración, inicialización y validación.
4. **Falta de modularización:** Lógica de SQL inline en memory_manager.py (10+ consultas dinámicas).

3. Análisis de Seguridad (Bandit + Revisión Manual)

3.1 Resumen de Hallazgos por Severidad

Severidad	Cantidad	Confianza Alta	Confianza Media
High	7	5	2
Medium	26	11	15
Low	55	0	55
Total	88	16	72

3.2 Vulnerabilidades Críticas y Altas

VULN-001: Terminal PTY - Ejecución Remota de Código (CRÍTICA)

- **Archivo:** `api/terminal.py` (líneas 162-183, 186-209)
- **CWE:** CWE-78, CWE-250
- **CVSS:** 9.8
- **Estado:** SIN CORREGIR (8+ meses)
- **Evidencia:**

Línea 187-209

```
master_fd, slave_fd = pty.openpty()
pid = os.fork()
if pid == 0:
    os.setsid()
    fcntl.ioctl(slave_fd, termios.TIOCSCTTY, 0)
    os.dup2(slave_fd, 0)
    os.dup2(slave_fd, 1)
    os.dup2(slave_fd, 2)
    os.close(master_fd)
    os.close(slave_fd)
    clean_env = {...}
    os.execve(DEFAULT_SHELL, [DEFAULT_SHELL, "--login"],
              clean_env)
```

- **Riesgo:** Cualquier usuario autenticado puede obtener shell interactiva completa del servidor.
- **Mitigación parcial observada:** Se agregó `validate_pty_command()` en línea 161, pero:
 - No hay allowlist estricta visible en la porción revisada.
 - No hay sandboxing (Docker, nsjail, chroot).
 - No hay auditoría de comandos.
 - El fork sigue siendo un proceso hijo sin aislamiento real.

VULN-002: Secretos por Defecto (CRÍTICA)

- **Archivo:** `core/config.py`
- **CWE:** CWE-798
- **Estado:** SIN CORREGIR (8+ meses)
- **Hallazgo Bandit:** Valores por defecto inseguros para `JWT_SECRET_KEY`, `ADMIN_SECRET`, `DB_ENCRYPTION_KEY`.
- **Riesgo:** Despliegues con secretos conocidos permiten falsificación de tokens y acceso administrativo.

VULN-003: Uso de MD5 para Seguridad (ALTA)

- **Archivos:** api/onlyoffice.py (líneas 577, 657), core/agent.py (líneas 1152, 2929), core/deep_researcher_utils.py (línea 360), core/tts_manager.py (línea 71)
- **CWE:** CWE-327
- **Bandit:** B324 (Confidence: High)
- **Riesgo:** Colisiones MD5 permiten predecir claves de documento y hashes de caché.
- **Ejemplo:**

```
# api/onlyoffice.py:577
key = md5(f"{doc.id}-{
    {doc.updated_at.isoformat()}}".encode()).hexdigest()
```

VULN-004: SSL Verification Deshabilitada (ALTA)

- **Archivo:** api/onlyoffice.py (línea 1159)
 - **CWE:** CWE-295
 - **Bandit:** B501 (Confidence: High)
 - **Hallazgo:**
- ```
async with httpx.AsyncClient(verify=False, timeout=300.0) as
 client:
 resp = await client.get(download_url)
```
- **Riesgo:** Ataques MITM al descargar documentos de OnlyOffice.

### VULN-005: Inyección SQL Potencial (MEDIA-ALTA)

- **Archivos:** api/chat.py, api/knowledge\_graph.py, core/memory\_manager.py (10 ocurrencias)
- **CWE:** CWE-89
- **Bandit:** B608 (Confidence: Low-Medium)
- **Hallazgo:** Construcción dinámica de consultas SQL con f-strings en core/memory\_manager.py:

```
core/memory_manager.py:897
update_query = f"""
 UPDATE langchain_pg_embedding
 SET {"", ".join(update_clauses)}
```

```
WHERE {" AND ".join(where_clauses)}
"""
```

- **Riesgo:** Si `update_clauses` o `where_clauses` contienen entrada de usuario sin sanitizar, hay inyección SQL.

#### **VULN-006: XML External Entity (XXE) (MEDIA)**

- **Archivo:** `api/caldav.py` (línea 264)
- **CWE:** CWE-20
- **Bandit:** B314 (Confidence: High)
- **Hallazgo:**

```
root_el = ET.fromstring(body)
```

- **Riesgo:** Parsing de XML sin `defusedxml` permite XXE.

#### **VULN-007: Binding a Todas las Interfaces (MEDIA)**

- **Archivo:** `api/main.py` (línea 553)
- **CWE:** CWE-605
- **Bandit:** B104 (Confidence: Medium)
- **Hallazgo:**

```
uvicorn.run("api.main:app", host="0.0.0.0", port=8000,
 reload=True)
```

- **Riesgo:** Exposición innecesaria en modo desarrollo.

#### **VULN-008: Directorio Temporal Hardcodeado (MEDIA)**

- **Archivos:** `api/main.py`, `core/config.py`, `core/tts_manager.py`, `api/terminal.py`
- **CWE:** CWE-377
- **Bandit:** B108 (Confidence: Medium)
- **Hallazgo:** Uso de `/tmp` sin `tempfile.mkdtemp()` ni permisos restrictivos.

#### **VULN-009: Request sin Timeout (MEDIA)**

- **Archivo:** `api/telegram.py` (línea 47)
- **CWE:** CWE-400
- **Bandit:** B113 (Confidence: Low)

- **Hallazgo:**

```
response = requests.post(telegram_api_url, json=payload)
```

## VULN-010: Hugging Face sin Revision Pinning (MEDIA)

- **Archivo:** core/reranker.py (líneas 59-60)

- **CWE:** CWE-494

- **Bandit:** B615 (Confidence: High)

- **Hallazgo:**

```
self._tokenizer =
 AutoTokenizer.from_pretrained(self.model_name)
self._model =
 AutoModelForSequenceClassification.from_pretrained(self.model_name)
```

## 3.3 Hallazgos Adicionales de Seguridad

- **Comandos peligrosos en PATH:** core/agent.py y core/autonomous\_heartbeat.py contienen lógica de ejecución de comandos.
- **Falta de rate limiting** en endpoints sensibles (terminal, OnlyOffice callback).
- **Falta de CORS estricto** en configuración de FastAPI.
- **Logging de información sensible** en core/config.py (W1203: f-strings en logging).

---

# 4. Análisis de Patrones y Arquitectura

## 4.1 Duplicación de Código

**OnlyOffice Config Generation:** - api/onlyoffice.py líneas 565-615 y 646-695: **Duplicación del 90%** en generación de configuración OnlyOffice. - **Impacto:** Mantenimiento doble, riesgo de inconsistencia.

**SQL Query Builders:** - core/memory\_manager.py tiene **10+ patrones similares** de construcción de consultas dinámicas.

## 4.2 God Objects y Clases Monolíticas

| Archivo       | Líneas | Responsabilidades                                         |
|---------------|--------|-----------------------------------------------------------|
| core/agent.py | 3,000+ | Orquestación LLM, memoria, tools, heartbeat, sanitización |

| Archivo                | Líneas | Responsabilidades                                  |
|------------------------|--------|----------------------------------------------------|
| core/config.py         | 1,500+ | Configuración, validación, logging, TTS, secretos  |
| core/memory_manager.py | 2,500+ | CRUD memoria, búsqueda semántica, metadata, topics |

## 4.3 Code Smells Adicionales

1. **Feature Envy:** Lógica deSQL en `memory_manager.py` que debería estar en un repositorio dedicado.
2. **Data Clumps:** Configuración de OnlyOffice repetida en múltiples funciones.
3. **Magic Strings:** Rutas hardcodeadas (`/api/onlyoffice/download/`, `/tmp/pollinations_images`).
4. **Primitive Obsession:** Uso de strings para IDs de documento en lugar de UUIDs tipados.

## 5. Auditoría de Dependencias y Configuración

### 5.1 Dependencias Críticas

- **FastAPI:** Framework principal, requiere actualización de seguridad.
- **LangChain:** Uso intensivo en `core/agent.py` y `core/llm_manager.py`.
- **PostgreSQL + pgvector:** Dependencia crítica para memoria semántica.
- **Neo4j:** Knowledge graph, sin evidencia de auditoría de Cypher injection.

### 5.2 Configuración de Seguridad

- **JWT:** Sin evidencia de validación de algoritmo en decodificación.
- **CORS:** Configuración no revisada en detalle.
- **Rate Limiting:** No implementado en endpoints críticos.

## 6. Recomendaciones Prioritarias

### 6.1 Acciones Inmediatas (P0 - Esta Semana)

| # | Acción                                         | Archivo                | Esfuerzo |
|---|------------------------------------------------|------------------------|----------|
| 1 | Reemplazar MD5 por SHA-256 en onlyoffice.py    | api/onlyoffice.py      | Bajo     |
| 2 | Habilitar verify=True en httpx                 | api/onlyoffice.py:1159 | Bajo     |
| 3 | Agregar timeout a requests.post                | api/telegram.py:47     | Bajo     |
| 4 | Implementar allowlist estricta en terminal PTY | api/terminal.py        | Alto     |
| 5 | Forzar cambio de secretos por defecto          | core/config.py         | Bajo     |
| 6 | Agregar defusedxml en CalDAV                   | api/caldav.py          | Bajo     |

### 6.2 Acciones a Corto Plazo (P1 - Próximas 2 Semanas)

| #  | Acción                                               | Archivo                            | Esfuerzo |
|----|------------------------------------------------------|------------------------------------|----------|
| 7  | Refactorizar call_model_node (CC 313)                | core/agent.py                      | Alto     |
| 8  | Extraer configuración OnlyOffice a función única     | api/onlyoffice.py                  | Medio    |
| 9  | Implementar prepared statements en memory_manager.py | core/memory_manager.py             | Alto     |
| 10 | Agregar rate limiting en terminal y OnlyOffice       | api/terminal.py, api/onlyoffice.py | Medio    |

| #  | Acción                                         | Archivo        | Esfuerzo |
|----|------------------------------------------------|----------------|----------|
| 11 | Implementar logging estructurado sin f-strings | core/config.py | Bajo     |
| 12 | Agregar tests de seguridad para terminal PTY   | tests/         | Medio    |

### 6.3 Acciones Estratégicas (P2 - Próximo Mes)

| #  | Acción                                                      | Esfuerzo |
|----|-------------------------------------------------------------|----------|
| 13 | Dividir core/agent.py en módulos especializados             | Alto     |
| 14 | Implementar CI/CD con análisis estático obligatorio         | Medio    |
| 15 | Auditoría de dependencias (pip-audit, npm audit)            | Medio    |
| 16 | Implementar sandboxing para terminal (Docker, nsjail)       | Alto     |
| 17 | Establecer política de secretos (HashiCorp Vault, env vars) | Medio    |

## 7. Módulos Recomendados para Reestructuración

### 7.1 core/agent.py

- **Extraer:** LLMOrchestrator, MemoryManager, ToolRegistry, HeartbeatManager
- **Objetivo:** Reducir CC de 313 a <50 por función.

### 7.2 core/memory\_manager.py

- **Extraer:** SQLQueryBuilder, SemanticSearchService, MetadataService
- **Objetivo:** Eliminar SQL inline y centralizar acceso a datos.

## 7.3 api/onlyoffice.py

- **Extraer:** OnlyOfficeConfigBuilder, OnlyOfficeJWTService
  - **Objetivo:** Eliminar duplicación del 90% en generación de config.
- 

## 8. Próximos Pasos

1. **Validar** con el equipo las recomendaciones P0.
  2. **Crear issues** en el tracker para cada vulnerabilidad High.
  3. **Asignar** responsables y fechas límite.
  4. **Implementar** fixes P0 en sprint actual.
  5. **Planificar** refactor P1 para próximo sprint.
- 

## 9. Anexos

### A. Herramientas Utilizadas

- radon (complejidad, mantenibilidad, raw, halstead)
- bandit (seguridad estática)
- grep / find (detección de patrones)
- read\_file\_tool (revisión manual)

### B. Archivos Críticos Revisados

- api/terminal.py
- api/onlyoffice.py
- core/agent.py
- core/config.py
- core/memory\_manager.py
- api/main.py
- api/chat.py
- api/knowledge\_graph.py

### C. Referencias

- CWE-78: OS Command Injection
- CWE-89: SQL Injection
- CWE-295: Improper Certificate Validation
- CWE-327: Use of a Broken or Risky Cryptographic Algorithm
- CWE-798: Use of Hard-coded Credentials